

**Universidade Federal do Vale do São Francisco –UNIVASF
Colegiado de Engenharia Elétrica**

Sistema de Monitoramento e Impressão de Dados de Joystick Analógico com a Ferramenta Pico SDK

Docente: Dr. Ricardo Menezes Prates

Discentes: Natan Éder Valeriano de Souza, Jônatas de Sousa Soares da Cruz

Disciplina: Microcontroladores e Microprocessadores (ELET00210)

Turma: E5

JUAZEIRO- BAHIA-BRASIL

JUNHO DE 2026

1. INTRODUÇÃO

Os sistemas embarcados modernos exigem alta eficiência no processamento de sinais e na comunicação com periféricos externos, minimizando o uso de pinos físicos e o consumo de energia. No cenário atual da engenharia de computação e automação, o microcontrolador RP2040, desenvolvido pela Raspberry Pi, destaca-se por sua arquitetura ARM Cortex-M0+ de núcleo duplo e pela versatilidade de seus periféricos internos. Para que um sistema embarcado interaja de forma eficaz com o mundo físico, ele depende diretamente de dois processos fundamentais: a conversão analógico-digital (ADC), que quantifica grandezas contínuas (como o deslocamento mecânico de um transdutor), e os protocolos de comunicação serial, responsáveis por transmitir dados processados para interfaces visuais. Esta prática laboratorial aborda a integração desses dois conceitos através da ferramenta educacional BitDogLab. Utiliza-se um joystick analógico como dispositivo de entrada de sinal contínuo e um display OLED de 0,96 polegadas baseado no controlador SSD1306 como interface de saída de dados. A comunicação entre o processador e a tela de exibição é realizada por meio do protocolo síncrono bidirecional I²C, que necessita de apenas duas linhas de sinal: SDA e SCL. Todo o desenvolvimento do firmware baseia-se na linguagem C com o auxílio do ecossistema Pico SDK, utilizando o ambiente do Visual Studio Code integrado ao simulador eletrônico Wokwi para fins de validação e depuração lógica.

2. OBJETIVOS

2.1. Objetivo Geral

Projetar, implementar e validar um sistema embarcado utilizando o microcontrolador RP2040 e a ferramenta Pico SDK para realizar a leitura analógica do eixo X de um joystick, processar os dados de conversão e exibir a tensão elétrica correspondente em tempo real em um display OLED via barramento I²C

2.2. Objetivos Específicos

- Configurar o periférico interno de conversão analógico-digital (ADC) do chip RP2040 para operar no canal de leitura associado ao pino de sinal do joystick.
- Implementar um driver de controle de baixo nível para o display OLED SSD1306, estabelecendo a sequência correta de comandos de inicialização, mapeamento de memória e desenho de caracteres.
- Desenvolver a lógica matemática necessária para converter o valor digital discreto de 12 bits gerado pelo ADC na magnitude real de tensão elétrica (na escala de 0,00 V a 3,30 V).
- Configurar o barramento de comunicação I²C para operar em modo mestre com taxa de transmissão em alta velocidade (400 kHz).

- Estabelecer uma rotina de temporização estável para garantir um tempo de amostragem e atualização periódica dos dados na tela a cada 500 ms, conforme os requisitos estabelecidos no projeto.
- Utilizar o simulador Wokwi integrado ao VS Code para realizar testes de software e conectividade lógica antes da gravação final no hardware físico.

3. MATERIAIS E MÉTODOS

3.1. Materiais Utilizados

O projeto foi desenvolvido inteiramente em ambiente de simulação computacional. Os recursos de hardware simulados e os softwares utilizados foram:

Hardware:

- Microcontrolador Raspberry Pi Pico W (baseado no chip RP2040).
- Placa de desenvolvimento educacional BitDogLab.
- Módulo de Joystick analógico integrado (eixo X conectado ao pino GPIO 26 / Canal 0 do ADC).
- Display OLED de 0,96 polegadas (resolução de 128x64 pixels, comunicação I²C).

Software:

- Ambiente de Desenvolvimento Comercial VS Code.
- Extensão oficial do Raspberry Pi Pico instalada no editor.
- Ferramenta de automação de compilação CMake associada ao gerenciador de build Ninja.
- Extensão do simulador eletrônico integrado Wokwi.

3.2 Metodologia

A execução do projeto seguiu três fases metodológicas sequenciais: configuração do ambiente e arquitetura de arquivos, modelagem matemática/lógica do firmware e simulação em ambiente virtual.

Na primeira fase, o arquivo de automação CMakeLists.txt foi estruturado para vincular os módulos nativos do Pico SDK ao código executável principal. Foram declaradas as dependências de hardware fundamentais: hardware_i2c e hardware_adc. Para possibilitar o funcionamento da tela, desenvolveu-se uma biblioteca modular de controle composta pelos arquivos ssd1306.c, ssd1306.h e font.h. O arquivo font.h mapeia uma matriz de pixels em formato hexadecimal para codificar caracteres alfanuméricos ASCII de tamanho 5x8 pixels. O arquivo ssd1306.c implementa os comandos do fabricante para ligar a bomba de carga interna (*charge pump*), definir o

modo de endereçamento horizontal de memória e realizar o envio em bloco do buffer de tela (1024 bytes) via escrita por I2C bloqueante.

Na segunda fase, implementou-se a rotina central no arquivo principal. O fluxo inicial do firmware executa a função padrão de inicialização de entrada e saída `stdio_init_all()`. Em seguida, configura os pinos físicos GPIO 14 (SDA) e GPIO 15 (SCL) da placa BitDogLab na porta periférica `i2c1`, definindo a frequência do clock de sincronismo em 400 kHz e habilitando os resistores internos de *pull-up*. O periférico analógico é iniciado por meio do comando `adc_init()`, configurando o pino GPIO 26 e selecionando-o como a fonte ativa de leitura.

Para a validação virtual do circuito nas conexões da placa BitDogLab, estruturou-se o arquivo descritor de hardware `diagram.json`, ligando a alimentação de 3.3V, as linhas comuns de aterramento (GND), o pino de leitura analógica horizontal ao GPIO 26 do microcontrolador e os pinos do barramento de dados seriais, mantendo os fios invisíveis na tela para otimização visual da simulação. O arquivo `wokwi.toml` foi criado para direcionar o simulador aos binários `.elf` e `.uf2` gerados automaticamente pelo compilador.

4. RESULTADOS E DISCUSSÕES

O comportamento do sistema embarcado foi monitorado de forma contínua através da janela de simulação interativa do Wokwi integrada no VS Code, apresentando estabilidade e total conformidade com os requisitos de projeto descritos no roteiro do laboratório.

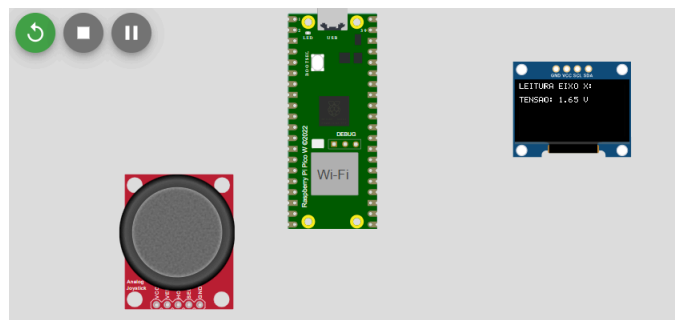
Com o joystick virtual em sua posição centralizada (condição de repouso mecânico), os potenciômetros internos formam um divisor de tensão simétrico. O nível de tensão resultante na entrada do canal 0 do ADC gerou uma palavra digital estável na metade da escala de quantização. Ao passar pela rotina matemática do firmware, o display OLED exibiu de forma precisa e centralizada o valor de 1,65 V, acompanhado do cabeçalho de identificação textual programado.

Durante os testes de deflexão do controle analógico ao longo do eixo horizontal (eixo X), observou-se as seguintes respostas dinâmicas na tela:

- Ao deslocar o joystick totalmente para o extremo correspondente ao limite superior de potencial e manter o cursor fixo, o conversor de 12 bits atingiu o teto de saturação, fazendo com que a tensão calculada e impressa no display OLED fosse atualizada para 3,30 V.
- Ao inverter o sentido do movimento mecânico, deslocando o componente para o extremo oposto, a tensão na entrada analógica decaiu até atingir o potencial de referência nulo (terra), resultando na exibição imediata do valor 0 V no visor.

A taxa de amostragem definida para 500 ms atendeu perfeitamente ao propósito prático de monitoramento sem causar sobrecarga ao processamento de dados do processador. Esse intervalo impediu a ocorrência de oscilações excessivamente rápidas nos dígitos exibidos na tela, eliminando o efeito de cintilação visual que comprometeria a leitura por um operador humano, enquanto manteve uma taxa de resposta ágil às alterações de posição do joystick. A transmissão de dados em bloco no barramento I2C operando a 400 kHz ocorreu sem corrupção de mensagens ou atrasos perceptíveis. A formatação de todas as strings de texto em letras maiúsculas solucionou problemas de deslocamento de índices no dicionário de fontes, garantindo a perfeita renderização gráfica de todos os caracteres alfanuméricos na interface física do display.

Figura 1: Leitura do eixo X do joystick em estado de repouso (1,65 V) exibida no display OLED virtual



Fonte: Autoria Própria (2026).

5. CONCLUSÃO

A realização desta atividade prática proporcionou uma sólida compreensão prática a respeito dos mecanismos de aquisição de sinais do mundo real por meio de conversores analógico-digitais, além da transmissão e exibição desses dados através de barramentos de comunicação serial síncrona. O microcontrolador RP2040 e os recursos nativos de hardware disponibilizados pelo Pico SDK provaram ser ferramentas robustas, ágeis e altamente eficientes para aplicações práticas de engenharia eletrônica, instrumentação e controle de processos. A inclusão do simulador eletrônico Wokwi de forma integrada ao ambiente de desenvolvimento do VS Code funcionou como uma valiosa ferramenta de suporte de engenharia. Essa integração reduziu expressivamente o ciclo de desenvolvimento e depuração do projeto, permitindo identificar e corrigir falhas de compilação, problemas de vinculação de arquivos de bibliotecas no CMake e inconsistências de renderização de caracteres em tela antes mesmo da implementação prática nos módulos físicos do laboratório. Desse modo, todos os requisitos obrigatórios estabelecidos pelo roteiro experimental foram plenamente cumpridos pela equipe, resultando em um protótipo de software estável, confiável e devidamente documentado de acordo com as especificações exigidas para a disciplina.

REFERÊNCIAS

RASPBERRY PI FOUNDATION. Raspberry Pi Pico W Datasheet. Cambridge: Raspberry Pi Ltd, 2022. Disponível em: <https://d229kd5ey79jzj.cloudfront.net/2717/pico-w-datasheet.pdf> . Acesso em 30 junho 2026.

WOKWI. *Wokwi - Online ESP32, Pi Pico, Arduino Simulator*. Disponível em: <https://wokwi.com/>. Acesso em: 30 junho 2026.